SQL in the LendingClub Risk Project

All queries ran in DuckDB — an embedded, serverless database in a local .duckdb file. Python's duckdb library executed SQL and returned results as pandas DataFrames.

1 Overall Default Rate

Count rows, sum binary is_default, divide for a percentage. Confirmed the 19.96% baseline used in every subsequent comparison.

```
SELECT COUNT(*) AS total,
SUM(is_default) AS defaults,
ROUND(100.0*SUM(is_default)
/COUNT(*),2) AS rate_pct
FROM loans
```

2 Default Rate by Grade

GROUP BY grade, order descending. Grade G (51.9%) defaults at 9× Grade A (5.8%) — sharpest single predictor in the dataset.

```
SELECT grade,
ROUND(100.0*SUM(is_default)
/COUNT(*),2) AS rate_pct
FROM loans
GROUP BY grade
ORDER BY rate_pct DESC
```

3 Avg & Median Interest Rate

AVG and MEDIAN side by side. Near-identical values confirmed symmetric distributions — rates range from 7.1% (Grade A) to 27.9% (Grade G).

```
SELECT grade,
ROUND(AVG(int_rate),2) AS avg,
ROUND(MEDIAN(int_rate),2) AS med
FROM loans
GROUP BY grade ORDER BY grade
```

4 Employment Length Bucketing

CASE WHEN collapsed 12 raw strings into 4 ordered buckets. A second CASE inside ORDER BY enforced chronological rather than alphabetical sort.

```
SELECT CASE
WHEN emp_length IN ('< 1 year',
'1 year') THEN '0-1 yr'
WHEN emp_length IN ('2 years',
'3 years') THEN '1-3 yr'
ELSE '5+ years' END AS bucket,
ROUND(100.0*SUM(is_default)
/COUNT(*),2) AS rate_pct
FROM loans GROUP BY bucket
```

5 Income Deciles — NTILE()

NTILE(10) ranked every borrower into one of 10 income bands. CTE separates the window step from the aggregation for readability.

```
WITH d AS (
SELECT is_default,
NTILE(10) OVER
(ORDER BY annual_inc) AS dec
FROM loans
)
SELECT dec,
ROUND(100.0*SUM(is_default)
/COUNT(*),2) AS rate_pct
FROM d GROUP BY dec ORDER BY dec
```

6 99th Percentile Cap

PERCENTILE_CONT computed the outlier ceiling for the loan-to-income ratio, passed into LEAST() in query 7 to cap extremes before K-Means clustering.

```
SELECT PERCENTILE_CONT(0.99)
WITHIN GROUP (
ORDER BY
CAST(loan_amnt AS DOUBLE)
/ NULLIF(CAST(
annual_inc AS DOUBLE),0)
) AS pct99
FROM loans
WHERE annual_inc IS NOT NULL
```

7 RFM Feature Engineering

Built all 3 clustering inputs in one query. COALESCE sets NULL delinquency → 999 (never missed = safest). NULLIF prevents ÷0. LEAST caps leverage at pct99.

```
SELECT
COALESCE(
mths_since_last_delinq,999)
AS R,
COALESCE(inq_last_6mths,0)
+COALESCE(delinq_2yrs,0) AS F,
LEAST(loan_amnt
/NULLIF(annual_inc,0),
) AS M
FROM loans WHERE annual_inc>0
```

8 Rebuild is_default (ALTER+UPDATE)

When is_default was lost during a Parquet save, it was rebuilt inside DuckDB without rerunning the pipeline. COPY wrote the fix back to disk.

```
ALTER TABLE loans
ADD COLUMN is_default INT;
UPDATE loans SET is_default=
CASE WHEN loan_status IN
('Charged Off','Default')
THEN 1 ELSE 0 END;
COPY loans TO 'file.parquet'
(FORMAT PARQUET);
```

SQL Techniques Reference

Technique	What it does	Where used in this project
`GROUP BY + SUM/COUNT`	Aggregate rows into buckets; count events	All default rate calculations across every query
`CASE WHEN`	Conditional if/else logic inside SQL	Bucketing emp_length, revol_util; rebuilding is_default
`COALESCE(val, fallback)`	Return first non-NULL value in a list	R feature: NULL delinquency → 999 (never missed = safe)
`NULLIF(a, b)`	Return NULL when a = b, preventing divide-by-zero	Leverage ratio: avoids ÷0 when annual_inc is zero
`LEAST(a, b)`	Return the smaller of two values	Capping loan-to-income ratio at the 99th-percentile value
`MEDIAN()`	Middle value; robust to outliers unlike AVG	Interest rate and DTI distribution comparisons
`PERCENTILE_CONT(p)`	Compute any percentile of a numeric column	99th-percentile ceiling for outlier removal before clustering
`NTILE(n) OVER (...)`	Window fn: assign each row a rank-bucket from 1 to n	Splitting annual_inc into 10 equal-sized deciles
`WITH ... AS (CTE)`	Name a sub-query for clean multi-step logic	Income decile query — separates NTILE from GROUP BY
`ALTER TABLE / UPDATE`	Add a column and populate it with conditional values	Rebuilding is_default after it was lost in Parquet save
`COPY ... TO (FORMAT PARQUET)`	Export a DuckDB table directly to Parquet on disk	Writing the repaired table to file without re-running Python